1D SDC - a Layered DAG

Jerry Kiely

October 2026

Questions

As I was implementing Ahmed's *minimum free energy* algorithm in Python, I started to consider the possibility that the calculated value may have come from a configuration different to the *target configuration*.

I wanted a way to see what actual configuration resulted in the *minimum free energy* at a specific temperature, to confirm that the calculated value came from the *target configuration*.

Is the *target configuration* always the most *thermodynamically favoured configuration* for all temperatures?

MFE

The *dynamic programming* approach to calculating the *minimum free energy* of a 1D SDC

- N positions
- $\leq k$ tiles per position
- $\mathcal{O}(Nk^2)$

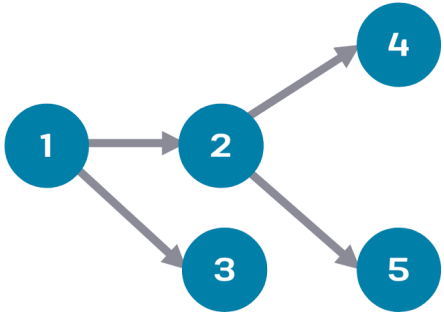
The algorithm calculates the *minimum free energy*, but does not provide the configuration that resulted in the *minimum free energy*. Therefore the calculation is a black box.

How do we know what configuration resulted in the calculated value of the *minimum free energy*, and if this configuration is indeed the *target configuration*?

DAG

A Directed Acyclic Graph

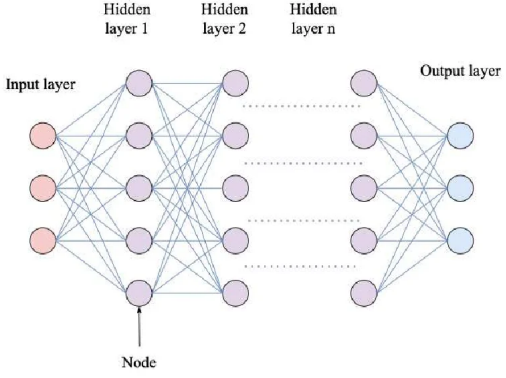
- Nodes
- Directed Edges
- No Cycles



Layered DAG

A Layered DAG

- Directed Acyclic Graph
- Partitioned into Layers
- Topological Ordering



Features of a Layered DAG

Why is a Layered DAG relevant to 1D SDC?

- Competing tiles occupy Layers (L_i)
- Layers are ordered ($L_i < L_j$ when $i < j$)
- A tile from L_i binds with a tile from L_{i+1}
- Edges connect tiles from layer i to layer $i + 1$
- Edges connect tiles with weight $\Delta G(T, r_i^m, l_{i+1}^n)$



MFE and Thermodynamic Favourability

For a Tileset represented as a Layered DAG, the Shortest Path gives you the most *thermodynamically favoured configuration* at a specific temperature.

If this configuration is the *target configuration*, then the Shortest Path is the *free energy* of the *target configuration*, and the *minimum free energy* of the Tileset.

Layered DAG Traversal

The Shortest Path through a Layered DAG

- Topological Sort
- Edge Relaxation

```
uv = self.get_edge_weight(u, v, T)
# relax edge u -> v
if dist[v] > dist[u] + uv:
    dist[v] = dist[u] + uv
    pred[v] = u
```

Note: for a Layered DAG, Layers induce a *topological ordering*, so sorting is unnecessary.

Comparison

Comparison of approaches:

	Time Complexity	Space Complexity
DP	$\mathcal{O}(Nk^2)$	$\mathcal{O}(k)$
DAG	$\mathcal{O}(V+E)$	$\mathcal{O}(V+E)$
Layered DAG	$\mathcal{O}(E)$	$\mathcal{O}(V+E)$

Note: For the Layered DAG, with respect to space complexity, we need E (the number of edges) space to hold the edges, and V (the number of vertices) space to hold the reconstructed path itself.

A Note on Edges

There are k_i competing tiles in L_i

- $k = \max_i(k_i)$ tiles per position
- k^2 possible bindings between tiles in consecutive positions
- N positions
- $(N-1)k^2$ possible bindings between tiles in all positions
- $\therefore$ no more than $(N-1)k^2$ edges
- $\therefore \mathcal{O}(E) = \mathcal{O}(Nk^2)$

MFE - Code

```python
def minimum_free_energy(tiles: List[List[Tile]], T: float) -> float:
    N = len(tiles)
    K = max(len(t) for t in tiles)

    M_curr: List[float] = [0] * K

    for j in range(len(tiles[0])):
        M_curr[j] = gibbs_m(tiles[0][j], T)

    for i in range(1, N):
        M_prev = M_curr
        M_curr = [0] * K

        for j in range(len(tiles[i])):
            M_temp = []

            for k in range(len(tiles[i - 1])):
                M_temp.append(M_prev[k] + gibbs_rl(tiles[i - 1][k], tiles[i][j], T))

            M_curr[j] = min(M_temp) + gibbs_m(tiles[i][j], T) + gibbs_assoc(T)

    return min(M_curr)
```

MFE - Test

```python
def test_minimum_free_energy__bitcopy_4pos_0():
    mfe = minimum_free_energy(read_sdc_tileset("./data/sdc/1D/bitcopy_04pos_0.json"), 273.15 + 37)
    assert math.isclose(mfe, -152.454, abs_tol = 0.001)

def test_minimum_free_energy__bitcopy_4pos_1():
    mfe = minimum_free_energy(read_sdc_tileset("./data/sdc/1D/bitcopy_04pos_1.json"), 273.15 + 37)
    assert math.isclose(mfe, -152.103, abs_tol = 0.001)



def test_minimum_free_energy__bitcopy_11pos_0():
    mfe = minimum_free_energy(read_sdc_tileset("./data/sdc/1D/bitcopy_11pos_0.json"), 273.15 + 37)
    assert math.isclose(mfe, -484.441, abs_tol = 0.001)

def test_minimum_free_energy__bitcopy_11pos_1():
    mfe = minimum_free_energy(read_sdc_tileset("./data/sdc/1D/bitcopy_11pos_1.json"), 273.15 + 37)
    assert math.isclose(mfe, -483.219, abs_tol = 0.001)
```

Shortest Path - Code

```python
def shortest_path(self, source: Tile, T: float) -> Tuple[float, List[Tile]]:
    pred = {}
    path = []

    dist = {n: float('inf') for n in self.get_nodes()}
    dist[source] = 0

    for l in range(len(self.tiles) - 1):
        for u, v in self.get_edge(l):
            uv = self.get_edge_weight(u, v, T)
            if dist[v] > dist[u] + uv:
                dist[v] = dist[u] + uv
                pred[v] = u

    node = self._dummy
    mfe  = dist[self._dummy]

    while node in pred:
        node = pred[node]
        mfe += self.get_node_weight(node, T) + gibbs_assoc(T)
        path.append(node)

    return mfe, path[::-1]
```

Shortest Path - Test

```python
def test_shortest_path__bitcopy_4pos_0():
    mfe, path = SDCGraph(read_sdc_tileset("./data/sdc/1D/bitcopy_04pos.json")).shortest_path(TA0, 273.15 + 37)
    assert math.isclose(mfe, -152.454, abs_tol = 0.001)
    assert path == [TA0, TB0, TC0, TDQ]

def test_shortest_path__bitcopy_4pos_1():
    mfe, path = SDCGraph(read_sdc_tileset("./data/sdc/1D/bitcopy_04pos.json")).shortest_path(TA1, 273.15 + 37)
    assert math.isclose(mfe, -152.103, abs_tol = 0.001)
    assert path == [TA1, TB1, TC1, TD]



def test_shortest_path__bitcopy_11pos_0():
    mfe, path = SDCGraph(read_sdc_tileset("./data/sdc/1D/bitcopy_11pos.json")).shortest_path(TA0, 273.15 + 37)
    assert math.isclose(mfe, -484.441, abs_tol = 0.001)
    assert path == [TA0, TB0, TC0, TD0, TE0, Tf0, TF0, TG0, TH0, TI0, TJ0, TKQ]

def test_shortest_path__bitcopy_11pos_1():
    mfe, path = SDCGraph(read_sdc_tileset("./data/sdc/1D/bitcopy_11pos.json")).shortest_path(TA1, 273.15 + 37)
    assert math.isclose(mfe, -483.219, abs_tol = 0.001)
    assert path == [TA1, TB1, TC1, TD1, TE1, Tf1, TF1, TG1, TH1, TI1, TJ1, TK]
```

Finally

Next steps?

- Multilayer Perceptron (MLP)
- Free Palestine!

Goodbye

Questions?